

基于 8051F310 单片机的垃圾分类系统设计

成绩

指导教师：罗志祥

专业班级：光电信息科学与工程 202409

姓名：陈恪瑾

学号：U202413925

联系电话：18571851291

1 任务要求



图 1: 工作场景：小区垃圾回收系统

1.1 实验目的

- 熟悉 C8051F310 单片机实验平台的 I/O 口、矩阵键盘、数码管、流水灯和蜂鸣器等外设资源的配置方法；
- 掌握 4 位数码管动态扫描显示、4×4 矩阵键盘扫描与消抖、74HCT164 串行驱动流水灯等常用接口程序设计方法；
- 理解定时器中断和外部中断的工作机制，能够利用 Timer0 构造 1ms 系统节拍，并在主循环状态机中完成倒计时、闪烁和报警等并发任务；

- 通过完整汇编程序的编写、构建与调试，提升对模块化程序结构、共享 RAM 状态变量和嵌入式系统故障定位方法的掌握程度。

1.2 实验内容

本设计基于 C8051F310EVM 实验平台，采用 MCS-51 汇编语言实现小区智能垃圾分类回收系统。系统以四位数码管显示四类垃圾桶剩余容量，以 F1-F4 分类按键完成开盖与投递，以 Timer0 和外部中断分别完成精确定时及提前关盖控制。具体要求如下。

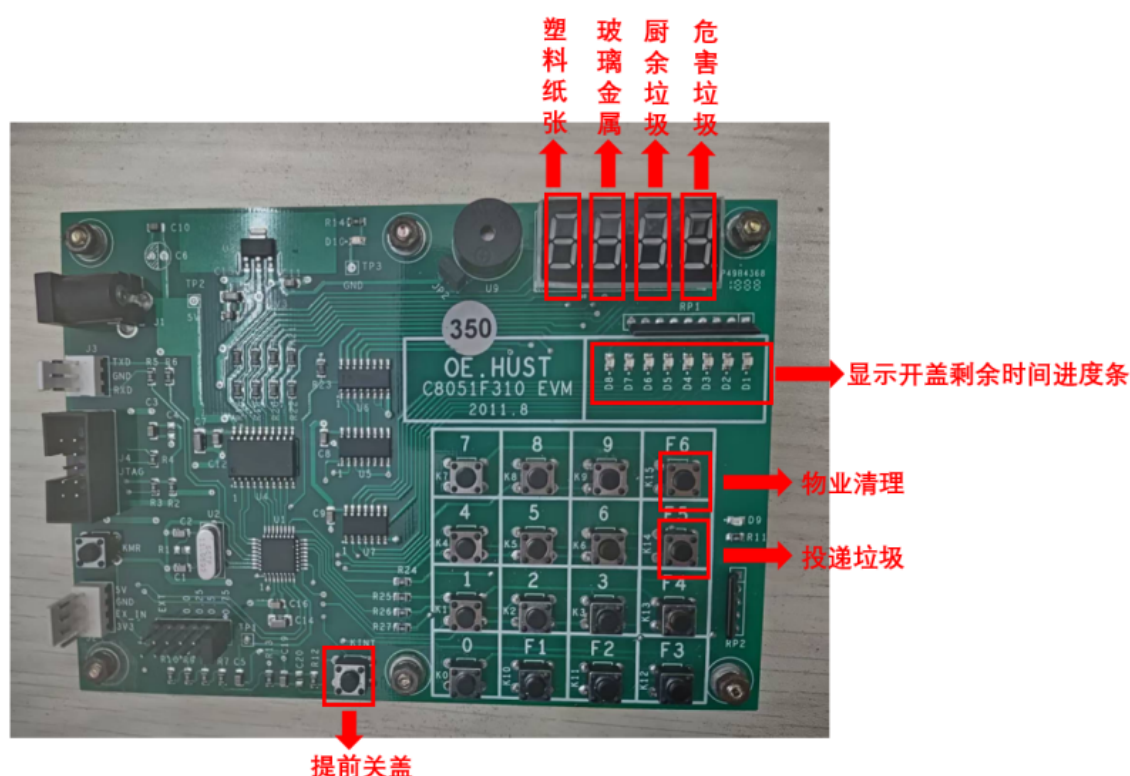


图 2: 开发板示意图

基础部分（满分 85）

- 系统将生活垃圾分为“塑料纸张”、“玻璃金属”、“厨余垃圾”和“危害垃圾”四类。上电初始化后，程序随机生成 4 个 0 ~ 9 的整数，分别作为四个垃圾桶的剩余可投放容量，并在四位数码管上同时显示。
- 用户按下 K10-K13 (F1-F4) 中的某一分类键选择对应垃圾桶。若该桶剩余容量为 0，则认为垃圾桶已满，蜂鸣器鸣响约 1s，对应数码管显示字母 F (FULL)，提示结束后恢复原容量显示，桶盖不开启。

- 若所选垃圾桶仍有剩余容量，则系统打开对应桶盖，D9 指示灯点亮，开盖时间设定为 8s；开盖期间，对应数码管以 5Hz 频率闪烁，显示该桶当前剩余容量。
- 在桶盖打开的 8s 内，用户再次按下同一个分类键表示投入 1 袋垃圾，系统将该桶容量减 1；该操作可重复执行，直至投放完成或容量减至 0。
- 桶盖计时由 Timer0 中断完成。程序以 1ms 为系统节拍维护 16 位倒计时变量，要求开盖 8s 后自动关闭桶盖，D9 指示灯熄灭，选中数码管停止闪烁。

1.3 提高部分（申优选做，85 分以上）

在基本功能基础上，当前程序进一步实现了以下扩展功能：

- 任一垃圾桶被选择后，四位数码管仍同时刷新显示全部垃圾桶状态；其中被选中的桶位通过 5Hz 闪烁突出显示，未选中的桶位保持正常容量显示。
- 桶盖开启期间，流水灯 D1-D8 作为 8s 倒计时进度条显示剩余开盖时间：开盖初始全亮，随后按秒级阈值逐格熄灭，倒计时结束后全部熄灭。
- 开盖期间支持 K1-K9 数字键批量投放。按下数字键 n 表示一次投入 n 袋垃圾；若当前容量足够，则容量立即减 n ；若投入袋数超过剩余容量，则蜂鸣器鸣响约 1s，对应数码管显示 E (ERROR)，并关闭桶盖拒绝本次投放。
- 若用户在 8s 内已完成投放，可按 Kint 外部中断键提前关盖。外部中断服务程序立即清除开盖状态和倒计时变量，使 D9 与 D1-D8 熄灭，系统返回待机状态。
- 增加物业维护功能：按下 K15 (F6) 后，四个垃圾桶容量统一恢复为 9，同时关闭当前开盖状态并鸣响蜂鸣器作为确认。
- 增加计算器个性化功能：按下 K14 (F5) 进入计算器模式，K0-K9 输入数字，K10-K13 分别表示加、减、乘、除，K15 (F6) 执行等号运算，K14 (F5) 用于退格或返回上一步，Kint 用于退出计算器模式并恢复垃圾分类系统。

2 设计思路

本系统基于 C8051F310 单片机实验平台，采用纯 MCS-51 汇编语言实现。整体程序按照“硬件初始化 + 主循环状态机 + Timer0 周期中断 + INT0 外部中断 + 独立外

设驱动模块”的结构组织，将垃圾分类业务、显示刷新、键盘扫描、流水灯输出和计算器扩展功能分离到不同源文件中，便于调试与后续维护。

系统的核心思想是：主循环只负责刷新显示、扫描键盘并根据当前状态执行一次性业务动作；所有需要精确定时的任务交由 Timer0 中断维护；所有需要快速响应的提前关盖动作交由外部中断完成。程序通过片内 RAM 中的状态变量保存四个垃圾桶容量、当前开盖桶号、提示覆盖桶号、错误覆盖桶号、倒计时计数值和计算器模式状态，从而避免长时间阻塞延时，使数码管显示、蜂鸣器报警和按键响应能够同时进行。

整体设计逻辑如下：

1. **系统初始化与随机容量生成模块**：程序复位后首先关闭看门狗，配置 24.5MHz 内部振荡器、端口输出模式、Timer0 工作方式和 INT0 下降沿触发方式。随后初始化片内 RAM 中的容量、键盘、显示、倒计时、报警和计算器状态变量。系统利用 Timer0 自由运行值与端口输入状态组合形成伪随机种子，经乘法、加法和除 10 取余运算后生成 CAP0~CAP3，作为四个垃圾桶的初始剩余容量。
2. **主循环轮询与业务状态机模块**：主循环首先调用流水灯更新函数，再连续刷新四位数码管以保持稳定显示，然后扫描 4×4 矩阵键盘。键盘返回值通过 KEYLAST 进行边沿检测，避免长按造成重复触发。在普通垃圾分类模式下，K10~K13 被解释为四类垃圾桶选择键，K1~K9 被解释为批量投放袋数，K14 进入计算器模式，K15 执行物业清空。主循环根据 LIDOPEN、SELBIN 和容量值判断当前应开盖、投放、拒投还是忽略无效按键。
3. **开盖与容量扣减模块**：当用户按下某一分类键且该桶容量不为 0 时，程序设置 LIDOPEN=1，记录当前桶号到 SELBIN，装入 8000ms 倒计时初值，并点亮 D9 表示桶盖打开。开盖期间，再次按下同一分类键将投放 1 袋垃圾；按下 K1~K9 则调用同一容量处理过程一次性投放指定袋数。若当前容量足够，程序直接扣减对应 CAP 变量；若容量不足，则设置 ERBIN 显示 E，启动 1s 蜂鸣提示，并立即关闭桶盖。
4. **Timer0 多任务系统节拍模块**：Timer0 工作在 16 位定时方式，按 1ms 周期重装初值。中断服务程序同时维护三类时间任务：一是递减 LIDHI:LIDLO 组成的 8s 开盖倒计时，计数归零后自动清除开盖状态并熄灭 D9；二是每 100ms 翻转一次 BLKFLG，为选中数码管提供 5Hz 闪烁节拍；三是递减 HINTHI:HINTLO 组成的 1s 提示倒计时，提示结束后关闭蜂鸣器并清除 F/E 覆盖显示。
5. **外部中断快速关盖模块**：INT0 与 Kint 按键相连，采用下降沿触发。普通模式下，若桶盖处于打开状态，外部中断服务程序立即清除 LIDOPEN、SELBIN 和倒计时变

量,并熄灭 D9,实现不依赖主循环扫描周期的提前关盖。若系统处于计算器模式,Kint 则被复用为退出键,用于清除计算器状态并回到垃圾分类模式。

6. **动态数码管显示与覆盖优先级模块:** 四位数码管采用分时动态扫描方式显示。普通模式下,各位分别显示 CAPO~CAP3;显示段码由 GET_SEG 统一决定。其覆盖优先级为:若当前位等于 ERRBIN 则显示 E,若等于 FULLBIN 则显示 F,若等于开盖桶号且 BLKFLG 处于熄灭相位则显示空白,否则显示正常容量数字。通过这种掩码覆盖方法,程序无需改变容量缓存即可完成错误、满桶和闪烁提示。
7. **流水灯倒计时进度条模块:** D1~D8 由 74HCT164 串行移位寄存器驱动,程序使用 LEDREG 保存当前输出掩码,使用 LASTBAR 保存上一次输出值。主循环根据 8000ms 倒计时剩余值映射出 0~8 个亮灯等级,只有掩码变化时才调用移位输出函数,从而减少无效刷新。桶盖关闭时流水灯强制全灭。
8. **提示报警与物业清空模块:** 当待机状态下选择容量为 0 的垃圾桶时,程序设置 FULLBIN 并启动蜂鸣器,使对应数码管显示 F 约 1s。按下 K15 (F6) 时,物业清空函数将四个容量变量统一恢复为 9,关闭当前开盖状态,清空流水灯,并鸣响蜂鸣器作为确认反馈。
9. **计算器个性化扩展模块:** 为体现功能扩展能力,程序加入独立计算器模式。按 K14 (F5) 进入计算器后,垃圾分类相关状态被暂停并清空提示输出,数码管转为显示计算器缓存 CALCD0~CALCD3。K0~K9 用于输入最多两位的操作数,K10~K13 分别表示加、减、乘、除,K15 执行等号计算,K14 用于退格或返回。计算完成后,结果通过同一套数码管动态扫描函数显示,Kint 可随时退出并恢复正常垃圾分类模式。

3 资源分配

3.1 I/O 口硬件资源分配

I/O 引脚	方向	功能定义	有效电平
P0.0	输出	D9 桶盖开启状态指示灯	低电平点亮
P0.1 / INT0	输入	Kint 外部中断键，用于提前关盖或退出计算器模式	下降沿触发
P0.6, P0.7	输出	四位数码管位选地址线 BA	00-11 选择第 0-3 位
P1.0~P1.7	输出	数码管段码输出 (dp,g,f,e,d,c,b,a 对应位序)	高电平点亮
P2.0~P2.3	输出	4×4 矩阵键盘行扫描线	当前扫描行拉低
P2.4~P2.7	输入	4×4 矩阵键盘列检测线	按下时为低电平
P3.1	输出	蜂鸣器控制信号	高电平鸣响
P3.3	输出	74HCT164 串行数据输入 DAT，用于 D1-D8 流水灯	逐位输出 LED 掩码
P3.4	输出	74HCT164 移位时钟 CLK，用于 D1-D8 流水灯	上升沿移位

3.2 寄存器资源分配

寄存器	功能定义
ACC(A)	主要累加器，用于键值判断、容量计算、段码查表、倒计时比较和中断临时运算
B	与 ACC 配合完成随机容量生成中的乘除法、计算器运算以及十进制拆分
DPTR	指向 ROM 查找表，包括数码管段码表、流水灯掩码表和键盘重映射表
R0	间接寻址指针或循环计数器，在容量数组访问与 74HCT164 移位输出中使用
R2	投放袋数参数，传递给 HANDLE_BAGS 判断容量是否足够
R3	临时保存当前垃圾桶编号，用于容量不足时恢复错误显示桶号
R4, R5, R6	数码管消隐延时、计算器结果拆位和短延时循环中的临时寄存器
R7	键盘消抖延时循环计数器
PSW	程序状态字，在中断服务程序中入栈保护，避免影响主程序状态
TH0, TL0	Timer0 高低字节寄存器，重装 0xF806 形成约 1ms 中断节拍
TMOD, TCON, IE	定时器工作方式、外部中断触发方式以及中断允许控制寄存器
P0MDOUT– P3MDOUT, P2MDIN, XBR1, OSCICN, PCA0MD	C8051F310 端口模式、交叉开关、内部振荡器和看门狗相关配置寄存器

3.3 片内 RAM 资源分配

存储单元	功能定义
30H~33H	CAP0~CAP3, 四个分类垃圾桶的剩余容量, 取值范围为 0 ~ 9
34H, 35H	KEYNOW, KEYLAST, 保存当前键号和上一轮键号, 用于按键边沿检测
36H	ROWIDX, 矩阵键盘扫描当前行号
37H, 38H	TMP, CNT, 通用临时变量和主循环刷新计数器
39H, 3BH	LEDREG, LASTBAR, 当前流水灯输出掩码和上一次输出掩码
42H, 43H	LIDLO, LIDHI, 16 位开盖倒计时, 单位为 ms, 初值为 8000ms
44H, 45H	BLKDIV, BLKFLG, 100ms 分频计数与 5Hz 闪烁相位标志
46H, 47H	HINTLO, HINTHI, 16 位蜂鸣器和 F/E 提示倒计时, 单位为 ms, 初值为 1000ms
4DH	LIDOPEN, 桶盖开启状态标志, 0 表示关闭, 1 表示打开
4EH	SELBIN, 当前被选中的垃圾桶编号, 0xFF 表示无选中桶
4FH	FULLBIN, 当前显示 F (FULL) 的桶号, 0xFF 表示无覆盖显示
50H	ERRBIN, 当前显示 E (ERROR) 的桶号, 0xFF 表示无覆盖显示
51H~53H	CALCMODE, CALCSTATE, CALCOP, 计算器模式、输入阶段和运算符状态
54H~57H	OP1, OP2, DIGCNT, CALCNEG, 计算器两个操作数、当前输入位数和负号标志
58H, 59H	RESLO, RESHI, 计算器 16 位结果缓存
5AH~5DH	CALCD0~CALCD3, 计算器模式下的四位数码管显示缓存, 支持数字、空白、负号和 E

4 流程图

系统运行流程可分为三个相互配合的部分：主程序负责垃圾分类业务状态判断和按键分发，Timer0 与 INT0 负责倒计时、闪烁、报警和提前关盖等实时任务，显示驱动负责四位数码管覆盖显示及 D1-D8 流水灯进度刷新。以下流程图分别给出这三部分的具体执行过程。

4.1 主程序状态轮询架构流程图

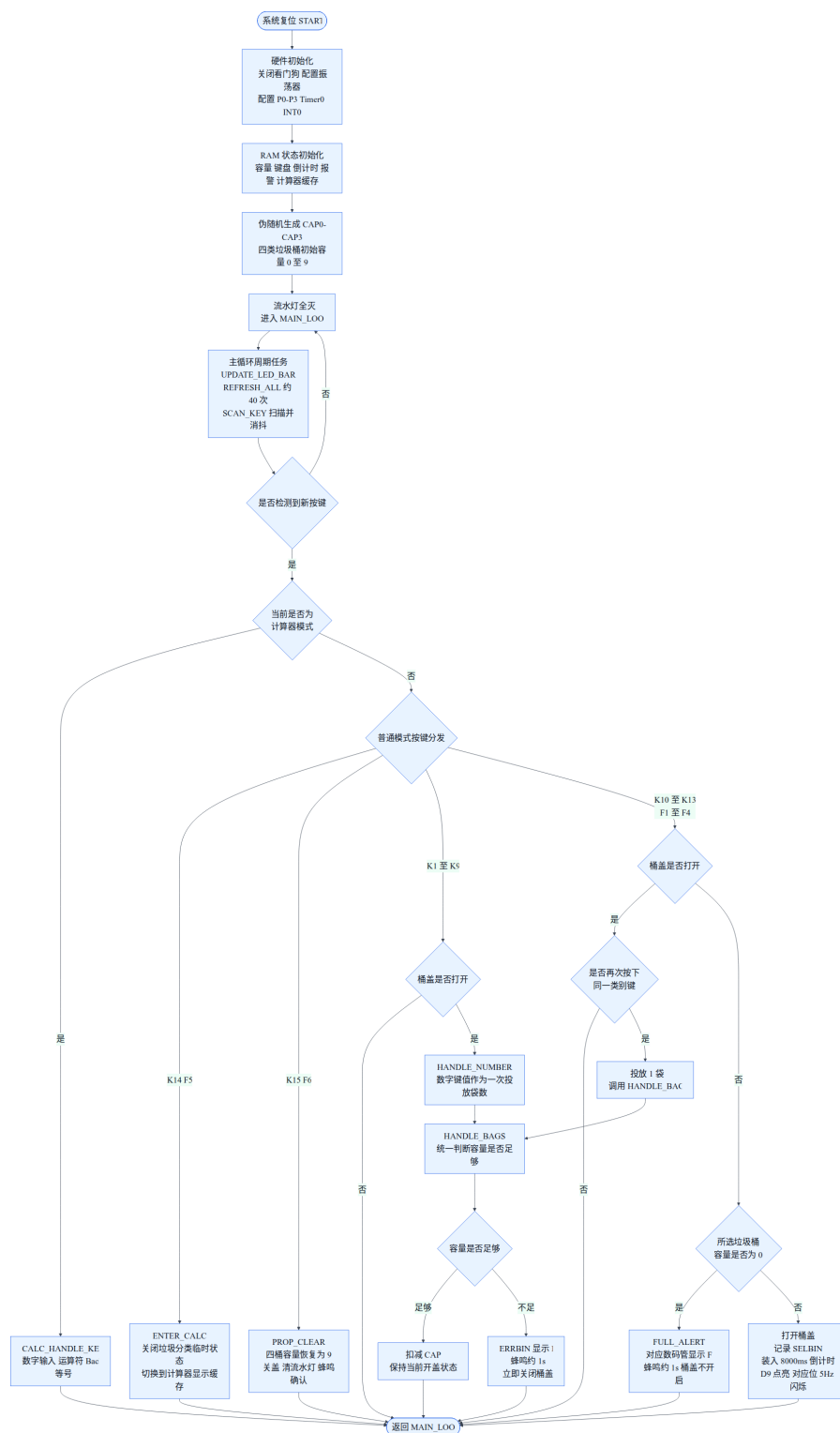


图 3: 主程序状态轮询与垃圾分类业务判别流程图

4.2 定时器/外部中断并行处理流程图

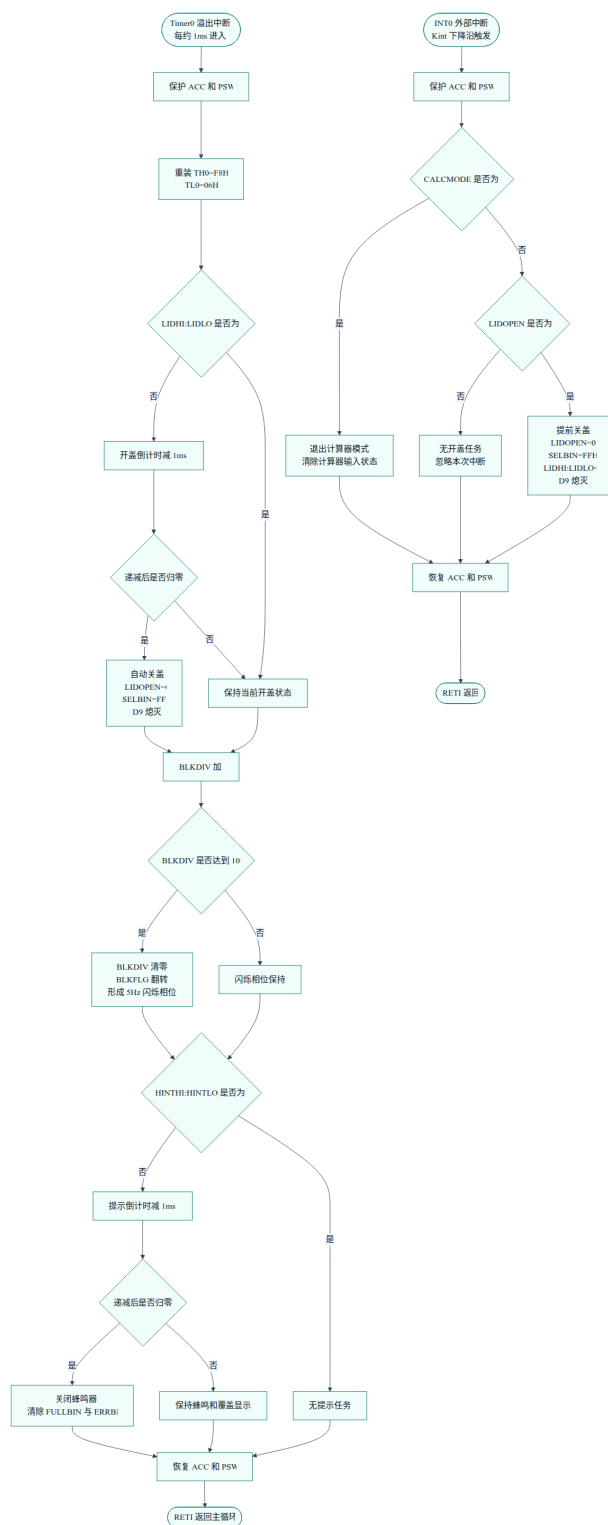


图 4: Timer0 系统节拍与 INT0 外部中断并行处理流程图

4.3 动态显示与流水灯刷新流程图

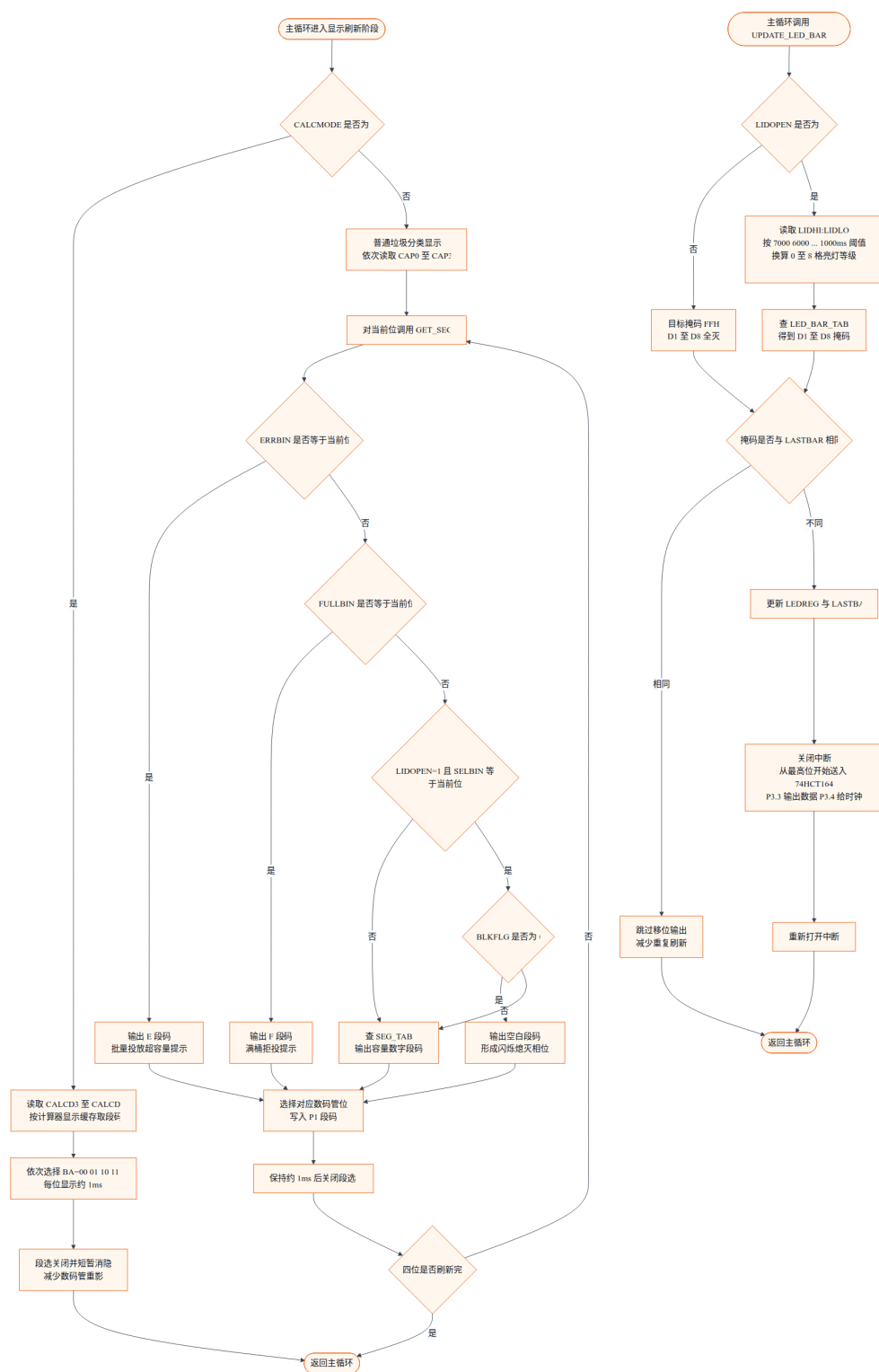


图 5: 数码管覆盖显示与 D1-D8 流水灯进度条刷新流程图

5 源代码（含文件头说明、语句行注释）

本系统源码采用多文件汇编结构组织。公共头文件定义特殊功能寄存器和片内 RAM 变量，启动文件完成硬件初始化，中断、显示、键盘、流水灯、查表、主状态机和计算器扩展分别放在独立模块中。各源文件开头均说明了模块用途、硬件连接或导出函数，关键语句处也保留了必要注释，便于结合前文设计思路阅读。

5.1 源码文件组织

文件	主要功能
inc/sfr.inc	C8051F310 标准及扩展特殊功能寄存器、位地址和硬件别名定义。
inc/iram.inc	片内 RAM 共享变量地址定义，包括容量、键盘、倒计时、提示和计算器缓存。
src/vectors.asm	复位向量、INT0 外部中断向量和 Timer0 中断向量。
src/init.asm	单片机初始化、端口模式配置、Timer0/INT0 配置、随机容量生成和程序入口。
src/isr.asm	Kint 提前关盖外部中断服务程序与 1ms Timer0 系统节拍中断服务程序。
src/display.asm	四位数码管动态扫描、F/E 覆盖显示、5Hz 闪烁显示和计算器显示缓存转换。
src/keypad.asm	4×4 矩阵键盘扫描、消抖和物理键位到逻辑键号的映射。
src/led_bar.asm	D1–D8 流水灯进度条计算和 74HCT164 串行移位输出。
src/tables.asm	数码管段码表、流水灯等级掩码表和键盘重映射表。
src/main.asm	主循环、垃圾桶开盖状态机、容量扣减、满桶/错误提示和物业清空。
src/calculator.asm	F5 进入的计算器扩展模式，包括输入、退格、四则运算和结果显示。

5.2 公共头文件

Listing 1: 特殊功能寄存器定义：inc/sfr.inc

```

1 ;-----
2 ; C8051F310 特殊功能寄存器定义 (sdas8051 语法)
3 ;-----
4
5 ; 标准 8051 特殊功能寄存器

```

```
6 P0      = 0x80
7 SP      = 0x81
8 DPL     = 0x82
9 DPH     = 0x83
10 PCON   = 0x87
11 TCON   = 0x88
12 TMOD   = 0x89
13 TL0    = 0x8A
14 TL1    = 0x8B
15 TH0    = 0x8C
16 TH1    = 0x8D
17 P1     = 0x90
18 SCON   = 0x98
19 SBUF   = 0x99
20 P2     = 0xA0
21 IE     = 0xA8
22 P3     = 0xB0
23 IP     = 0xB8
24 PSW    = 0xD0
25 ACC    = 0xE0
26 B      = 0xF0
```

```
27
```

```
28 ; P0 位地址
```

```
29 P0_0    = 0x80
30 P0_1    = 0x81
31 P0_2    = 0x82
32 P0_3    = 0x83
33 P0_4    = 0x84
34 P0_5    = 0x85
35 P0_6    = 0x86
36 P0_7    = 0x87
```

```
37
```

```
38 ; P3 位地址
```

```
39 P3_0    = 0xB0
40 P3_1    = 0xB1
41 P3_2    = 0xB2
42 P3_3    = 0xB3
43 P3_4    = 0xB4
44 P3_5    = 0xB5
45 P3_6    = 0xB6
46 P3_7    = 0xB7
```

```
47
```

```
48 ; IE 中断允许寄存器位地址
```

```
49 EX0     = 0xA8
50 ET0     = 0xA9
51 EX1     = 0xAA
52 ET1     = 0xAB
53 ES      = 0xAC
54 ET2     = 0xAD
```

```

55 EA      = 0xAF
56
57 ; TCON 定时/中断控制寄存器位地址
58 IT0     = 0x88
59 IE0     = 0x89
60 IT1     = 0x8A
61 IE1     = 0x8B
62 TR0     = 0x8C
63 TF0     = 0x8D
64 TR1     = 0x8E
65 TF1     = 0x8F
66
67 ; C8051F310 扩展特殊功能寄存器
68 PCA0MD  = 0xD9
69 POMDOUT = 0xA4
70 P1MDOUT = 0xA5
71 P2MDOUT = 0xA6
72 P3MDOUT = 0xA7
73 P2MDIN  = 0xF3
74 XBR1    = 0xE2
75 OSCICN  = 0xB2
76
77 ; 便于阅读的硬件别名
78 D9_LED  = P0_0      ; P0.0, 低电平点亮
79 BUZZER  = P3_1      ; P3.1, 高电平鸣响
80 LEDDAT  = P3_3      ; P3.3, 74HCT164 数据线
81 LEDCLK  = P3_4      ; P3.4, 74HCT164 时钟线

```

Listing 2: 片内 RAM 共享变量定义: inc/iram.inc

```

1 ;-----
2 ; 内部 RAM 符号定义 (各模块共享)
3 ;-----
4
5 ; 四个垃圾桶剩余容量 (每个 0-9)
6 CAP0    = 0x30
7 CAP1    = 0x31
8 CAP2    = 0x32
9 CAP3    = 0x33
10
11 ; 键盘扫描状态
12 KEYNOW  = 0x34
13 KEYLAST = 0x35
14 ROWIDX  = 0x36
15
16 ; 通用临时变量和循环计数
17 TMP     = 0x37
18 CNT     = 0x38
19

```

```

20 ; 流水灯状态 (D1-D8 经 74HCT164 驱动, 1=灭, 0=亮)
21 LEDREG = 0x39
22 LASTBAR = 0x3B
23
24 ; 桶盖倒计时 (16 位, 单位 ms, 由 Timer0 中断递减)
25 LIDLO = 0x42
26 LIDHI = 0x43
27
28 ; 5 Hz 闪烁状态
29 BLKDIV = 0x44 ; 每 1ms 递增, 到 100 后清零
30 BLKFLG = 0x45 ; 每 100ms 翻转一次 (0 或 1)
31
32 ; 报警倒计时 (16 位, 单位 ms, 用于驱动蜂鸣器约 1 秒)
33 HINTLO = 0x46
34 HINTHI = 0x47
35
36 ; 系统状态
37 LIDOPEN = 0x4D ; 0=关闭, 1=打开
38 SELBIN = 0x4E ; 当前打开的桶号 (0-3), 0xFF=无
39 FULLBIN = 0x4F ; 当前显示 F 的桶号, 0xFF=无
40 ERBBIN = 0x50 ; 当前显示 E 的桶号, 0xFF=无
41
42 ; 计算器模式状态
43 CALCMODE = 0x51 ; 0=垃圾投放模式, 1=计算器模式
44 CALCSTATE = 0x52 ; 0=输入操作数1, 1=已选运算符, 2=输入操作数2, 3=显示结果
45 CALCOP = 0x53 ; 1=加, 2=减, 3=乘, 4=除
46 OP1 = 0x54
47 OP2 = 0x55
48 DIGCNT = 0x56 ; 当前操作数已输入位数 (0-2)
49 CALCNEG = 0x57 ; 结果是否为负
50 RESLO = 0x58
51 RESHI = 0x59
52 CALCD0 = 0x5A ; 计算器显示缓存: 0-9 数字, 0x0A=空白, 0x0B='-', 0x0C='E'
53 CALCD1 = 0x5B
54 CALCD2 = 0x5C
55 CALCD3 = 0x5D

```

5.3 启动与中断向量

Listing 3: 中断向量表: src/vectors.asm

```

1 ;-----
2 ; vectors.asm — 中断向量表与复位入口
3 ; 必须最先链接, 使代码从 0x0000 开始
4 ;-----
5     .include "sfr.inc"
6
7     .module vectors

```



```

8
9     .area VECTOR (ABS, CODE)
10
11     ; 复位向量 @ 0x0000
12     .org 0x0000
13     ljmp START
14
15     ; INTO (外部中断 0, Kint 提前关盖键) @ 0x0003
16     .org 0x0003
17     ljmp EXT0_ISR
18
19     ; Timer0 溢出中断 (1ms 节拍) @ 0x000B
20     .org 0x000B
21     ljmp TIMERO_ISR
22
23     ; Timer1 和 EXT1 未使用, 填充到 0x0100
24     .org 0x0100
25     ljmp START           ; 保护入口: 异常跳转到此处也重新启动

```

Listing 4: 硬件初始化与程序入口: src/init.asm

```

1 ;-----
2 ; init.asm — MCU 硬件初始化与程序入口
3 ;-----
4     .include "sfr.inc"
5     .include "iram.inc"
6
7     .module init
8
9     .globl START
10    .globl MAIN_LOOP
11
12    .area INIT_CODE (CODE)
13
14    START:
15        ; 关闭看门狗 (PCA0MD.6 = 0)
16        anl  PCA0MD, #0xBF
17        mov  PCA0MD, #0x00
18
19        ; 内部振荡器 24.5 MHz
20        mov  OSCICN, #0x83
21
22        ; 端口输出模式:
23        ;   P0: P0.0(D9) + P0.6/P0.7(数码管位选) 推挽输出
24        ;   P1: 全部推挽输出 (数码管段选)
25        ;   P2: P2.0-P2.3 推挽输出 (键盘行), P2.4-P2.7 开漏输入 (键盘列)
26        ;   P3: P3.1(蜂鸣器) + P3.3(DATA) + P3.4(CLK) 推挽输出
27        mov  POMDOUT, #0xC1      ; P0.7、P0.6、P0.0
28        mov  P1MDOUT, #0xFF

```

```

29     mov  P2MDOUT, #0x0F
30     mov  P3MDOUT, #0x1A      ; P3.4、P3.3、P3.1
31
32     ; P2 列输入使用数字输入
33     mov  P2MDIN,  #0xFF
34
35     ; 使能交叉开关
36     mov  XBR1, #0x40
37
38     ; 初始端口状态
39     setb D9_LED              ; D9 熄灭（低电平点亮）
40     clr  BUZZER              ; 蜂鸣器关闭
41     mov  P2, #0xFF           ; 所有键盘行释放为高电平
42
43     ; Timer0 模式 1（16 位），时钟为 SYSCLK/12
44     anl  TMOD, #0xF0
45     orl  TMOD, #0x01
46     mov  TH0, #0xF8          ; 24.5MHz/12 下 1ms 重装值
47     mov  TL0, #0x06
48
49     ; INTO 下降沿触发
50     setb ITO
51
52     ; 使能 Timer0 中断、INT0 中断和总中断
53     setb ETO
54     setb EX0
55     setb EA
56
57     ; 启动 Timer0
58     setb TRO
59
60     ; 初始化内部 RAM 状态
61     mov  LIDOPEN, #0x00
62     mov  SELBIN,  #0xFF
63     mov  FULLBIN, #0xFF
64     mov  ERRLIN,  #0xFF
65     mov  LIDL0,   #0x00
66     mov  LIDHI,   #0x00
67     mov  BLKDIV,  #0x00
68     mov  BLKFLG,  #0x00
69     mov  HINTLO,  #0x00
70     mov  HINTHI,  #0x00
71     mov  KEYLAST, #0xFF
72     mov  CALCMODE, #0x00
73     mov  CALCSTATE, #0x00
74     mov  CALCOP,   #0x00
75     mov  OP1,       #0x00
76     mov  OP2,       #0x00
77     mov  DIGCNT,    #0x00

```

```

78     mov  CALCNEG,    #0x00
79     mov  RESLO,      #0x00
80     mov  RESHI,      #0x00
81     mov  CALCD0,     #0x0A
82     mov  CALCD1,     #0x0A
83     mov  CALCD2,     #0x0A
84     mov  CALCD3,     #0x0A
85
86     ; 生成伪随机种子: 先刷新显示 60 次, 再读取 TLO~TH0~P2
87     mov  CNT, #60
88
89 SEED_SPIN:
90     lcall REFRESH_ALL
91     djnz CNT, SEED_SPIN
92
93     mov  A, TLO
94     xrl  A, TH0
95     xrl  A, P2
96
97     ; 用简单 LCG 步进从种子生成 CAP0..CAP3
98     mov  B, #10
99     div  AB
100    mov  CAP0, B          ; CAP0 = seed % 10
101
102    mov  A, B
103    mov  B, #17
104    mul  AB
105    add  A, #31
106    mov  B, #10
107    div  AB
108    mov  CAP1, B
109
110    mov  A, B
111    mov  B, #13
112    mul  AB
113    add  A, #7
114    mov  B, #10
115    div  AB
116    mov  CAP2, B
117
118    mov  A, B
119    mov  B, #11
120    mul  AB
121    add  A, #9
122    mov  B, #10
123    div  AB
124    mov  CAP3, B
125
126    ; 初始化流水灯 (全灭)

```

```

127     mov  LEDREG,  #0xFF
128     mov  LASTBAR, #0xFF
129     lcall SHIFT_LED_BAR
130
131     ljmp  MAIN_LOOP

```

5.4 中断服务程序

Listing 5: 外部中断与 Timer0 中断：src/isr.asm

```

1 ;-----
2 ; isr.asm — 中断服务程序
3 ;   EXT0_ISR  : Kint 提前关盖键 (INT0, 下降沿触发)
4 ;   TIMERO_ISR: 1ms 系统节拍 (Timer0 溢出)
5 ;-----
6     .include "sfr.inc"
7     .include "iram.inc"
8
9     .module isr
10
11     .globl EXT0_ISR
12     .globl TIMERO_ISR
13
14     .area ISR_CODE (CODE)
15
16 ;-----
17 ; EXT0_ISR — 提前关盖 (Kint 按键, P0.1)
18 ; 立即关闭桶盖: 清除 LIDOPEN, 复位倒计时, 熄灭 D9。
19 ;-----
20 EXT0_ISR:
21     push ACC
22     push PSW
23
24     mov  A, CALCMODE
25     jz   EXO_NORMAL
26
27     ; 计算器模式下, Kint 作为退出键。
28     mov  CALCMODE, #0x00
29     mov  CALCSTATE, #0x00
30     mov  CALCOP,   #0x00
31     mov  OP1,      #0x00
32     mov  OP2,      #0x00
33     mov  DIGCNT,   #0x00
34     mov  CALCNEG,  #0x00
35     sjmp EXO_DONE
36
37 EXO_NORMAL:
38     mov  A, LIDOPEN

```

```

39     jz    EXO_DONE          ; 桶盖已关闭时直接忽略
40
41     mov   LIDOPEN, #0x00
42     mov   SELBIN,  #0xFF
43     mov   LIDL0,    #0x00
44     mov   LIDHI,    #0x00
45     mov   BLKDIV,   #0x00
46     mov   BLKFLG,   #0x00
47     setb  D9_LED          ; D9 熄灭
48
49 EXO_DONE:
50     pop   PSW
51     pop   ACC
52     reti
53
54 ;-----
55 ; TIMER0_ISR — 1ms 系统节拍
56 ; 重装 TH0/TLO, 并维护桶盖倒计时、闪烁标志和蜂鸣/提示倒计时。
57 ;-----
58 TIMER0_ISR:
59     push  ACC
60     push  PSW
61
62     ; 重装 1ms 初值: 65536 - (24500000/12/1000) = 63494 = 0xF806
63     mov   TH0, #0xF8
64     mov   TLO, #0x06
65
66     ;--- 桶盖倒计时 (LIDHI:LIDL0, 单位 ms) ---
67     mov   A, LIDL0
68     orl   A, LIDHI
69     jz    TO_SKIP_LID
70
71     mov   A, LIDL0
72     jnz   TO_DEC_LO
73     dec   LIDHI
74 TO_DEC_LO:
75     dec   LIDL0
76
77     mov   A, LIDL0
78     orl   A, LIDHI
79     jnz   TO_SKIP_LID
80
81     ; 倒计时归零: 自动关盖
82     mov   LIDOPEN, #0x00
83     mov   SELBIN,  #0xFF
84     setb  D9_LED          ; D9 熄灭
85
86 TO_SKIP_LID:
87

```

```

88      ;--- 5Hz 闪烁标志（每 100ms 翻转一次） ---
89      inc BLKDIV
90      mov A, BLKDIV
91      cjne A, #100, TO_HINT
92      mov BLKDIV, #0x00
93      mov A, BLKFLG
94      xrl A, #0x01
95      mov BLKFLG, A
96
97 TO_HINT:
98      ;--- 报警倒计时（HINTHI:HINTLO, 单位 ms） ---
99      ; 归零后关闭蜂鸣器，并清除 F/E 覆盖显示
100     mov A, HINTLO
101     orl A, HINTHI
102     jz TO_END
103
104     mov A, HINTLO
105     jnz TO_DEC_HL
106     dec HINTHI
107 TO_DEC_HL:
108     dec HINTLO
109
110     mov A, HINTLO
111     orl A, HINTHI
112     jnz TO_END
113
114     ; 报警结束：关闭蜂鸣器，清除 F/E 显示
115     clr BUZZER
116     mov FULLBIN, #0xFF
117     mov ERRBIN, #0xFF
118
119 TO_END:
120     pop PSW
121     pop ACC
122     reti

```

5.5 外设驱动模块

Listing 6: 四位数码管动态显示驱动：src/display.asm

```

1 ;-----
2 ; display.asm — 4 位数码管显示驱动
3 ;
4 ; 硬件：LG3641AH 共阴数码管，段选高电平点亮
5 ; 段选：P1 (a=P1.7 .. dp=P1.0)
6 ; 位选：P0.7(B) P0.6(A)
7 ; BA=00 -> 第 0 位（最左，CAP0）
8 ; BA=01 -> 第 1 位（CAP1）

```

```

9 ;    BA=10 -> 第 2 位 (CAP2)
10 ;    BA=11 -> 第 3 位 (最右, CAP3)
11 ;
12 ; 导出函数:
13 ;    REFRESH_ALL    — 对 4 位数码管动态扫描一次
14 ;    DELAY_DIG      — 单位数码管保持约 1ms
15 ;
16 ; 使用: BLKFLG, SELBIN, LIDOPEN, ERBBIN, FULLBIN, CAP0..CAP3
17 ; 会改写: ACC, B, DPTR
18 ;-----
19     .include "sfr.inc"
20     .include "iram.inc"
21
22     .module display
23
24     .globl REFRESH_ALL
25     .globl DELAY_DIG
26
27     ; tables.asm 中定义的外部符号
28     .globl SEG_TAB
29
30     .area DISP_CODE (CODE)
31
32 ;-----
33 ; REFRESH_ALL — 显示 4 位数码管各一次 (每次键盘扫描前约调用 40 次)
34 ;
35 ; 垃圾桶容量显示顺序:
36 ;    BA=00 -> CAP0, BA=01 -> CAP1, BA=10 -> CAP2, BA=11 -> CAP3
37 ;-----
38 REFRESH_ALL:
39     mov     A, CALCMODE
40     jz      REFRESH_NORMAL
41     ljmp    REFRESH_CALC
42
43 REFRESH_NORMAL:
44     ; 第 0 位 (BA=00)
45     mov     A, CAP0
46     mov     TMP, #0
47     lcall   GET_SEG
48     mov     B, A
49     mov     P1, #0x00      ; 切换位选前先灭段, 避免鬼影
50     lcall   DISPLAY_BLANK_GAP
51     anl     P0, #0x3F      ; BA=00
52     lcall   DISPLAY_BLANK_GAP
53     mov     P1, B
54     lcall   DELAY_DIG
55     mov     P1, #0x00
56     lcall   DISPLAY_BLANK_GAP
57

```

```

58     ; 第 1 位 (BA=01)
59     mov  A, CAP1
60     mov  TMP, #1
61     lcall GET_SEG
62     mov  B, A
63     mov  P1, #0x00
64     lcall DISPLAY_BLANK_GAP
65     anl  P0, #0x3F
66     orl  P0, #0x40          ; BA=01
67     lcall DISPLAY_BLANK_GAP
68     mov  P1, B
69     lcall DELAY_DIG
70     mov  P1, #0x00
71     lcall DISPLAY_BLANK_GAP
72
73     ; 第 2 位 (BA=10)
74     mov  A, CAP2
75     mov  TMP, #2
76     lcall GET_SEG
77     mov  B, A
78     mov  P1, #0x00
79     lcall DISPLAY_BLANK_GAP
80     anl  P0, #0x3F
81     orl  P0, #0x80          ; BA=10
82     lcall DISPLAY_BLANK_GAP
83     mov  P1, B
84     lcall DELAY_DIG
85     mov  P1, #0x00
86     lcall DISPLAY_BLANK_GAP
87
88     ; 第 3 位 (BA=11)
89     mov  A, CAP3
90     mov  TMP, #3
91     lcall GET_SEG
92     mov  B, A
93     mov  P1, #0x00
94     lcall DISPLAY_BLANK_GAP
95     anl  P0, #0x3F
96     orl  P0, #0xC0          ; BA=11
97     lcall DISPLAY_BLANK_GAP
98     mov  P1, B
99     lcall DELAY_DIG
100    mov  P1, #0x00
101    lcall DISPLAY_BLANK_GAP
102
103    ret
104
105 REFRESH_CALC:
106    ; 逻辑第 3 位 (BA=00)

```



```

107     mov  A, CALCD3
108     lcall GET_CALC_SEG
109     mov  B, A
110     mov  P1, #0x00
111     lcall DISPLAY_BLANK_GAP
112     anl  P0, #0x3F
113     lcall DISPLAY_BLANK_GAP
114     mov  P1, B
115     lcall DELAY_DIG
116     mov  P1, #0x00
117     lcall DISPLAY_BLANK_GAP
118
119     ; 逻辑第 2 位 (BA=01)
120     mov  A, CALCD2
121     lcall GET_CALC_SEG
122     mov  B, A
123     mov  P1, #0x00
124     lcall DISPLAY_BLANK_GAP
125     anl  P0, #0x3F
126     orl  P0, #0x40
127     lcall DISPLAY_BLANK_GAP
128     mov  P1, B
129     lcall DELAY_DIG
130     mov  P1, #0x00
131     lcall DISPLAY_BLANK_GAP
132
133     ; 逻辑第 1 位 (BA=10)
134     mov  A, CALCD1
135     lcall GET_CALC_SEG
136     mov  B, A
137     mov  P1, #0x00
138     lcall DISPLAY_BLANK_GAP
139     anl  P0, #0x3F
140     orl  P0, #0x80
141     lcall DISPLAY_BLANK_GAP
142     mov  P1, B
143     lcall DELAY_DIG
144     mov  P1, #0x00
145     lcall DISPLAY_BLANK_GAP
146
147     ; 逻辑第 0 位 (BA=11)
148     mov  A, CALCD0
149     lcall GET_CALC_SEG
150     mov  B, A
151     mov  P1, #0x00
152     lcall DISPLAY_BLANK_GAP
153     anl  P0, #0x3F
154     orl  P0, #0xC0
155     lcall DISPLAY_BLANK_GAP

```

```

156     mov P1, B
157     lcall DELAY_DIG
158     mov P1, #0x00
159     lcall DISPLAY_BLANK_GAP
160
161     ret
162
163 ;-----
164 ; GET_SEG — 获取 TMP 指定位置的段码
165 ;   输入: A = 容量值 (0-9)
166 ;         TMP = 数码管位置 (0-3)
167 ;   输出: A = 写入 P1 的段码
168 ;
169 ; 覆盖显示优先级 (从高到低):
170 ;   1. ERRBIN == TMP -> 显示 E (批量投放错误)
171 ;   2. FULLBIN == TMP -> 显示 F (满桶提示)
172 ;   3. LIDOPEN 且 SELBIN == TMP 且 BLKFLG == 0 -> 熄灭形成闪烁
173 ;   4. 正常显示 SEG_TAB 中的数字段码
174 ;-----
175 GET_SEG:
176     push ACC                ; 保存容量值
177
178     ; 检查 E 覆盖显示
179     mov A, ERRBIN
180     cjne A, TMP, GS_CHECK_FULL
181     pop ACC
182     mov A, #0x9E            ; E 的段码
183     ret
184
185 GS_CHECK_FULL:
186     ; 检查 F 覆盖显示
187     mov A, FULLBIN
188     cjne A, TMP, GS_CHECK_BLINK
189     pop ACC
190     mov A, #0x8E            ; F 的段码
191     ret
192
193 GS_CHECK_BLINK:
194     ; 检查开盖闪烁的熄灭阶段
195     mov A, LIDOPEN
196     jz GS_NORMAL
197     mov A, SELBIN
198     cjne A, TMP, GS_NORMAL
199     mov A, BLKFLG
200     jnz GS_NORMAL
201     pop ACC
202     mov A, #0x00            ; 闪烁熄灭阶段显示空白
203     ret
204

```

```

205 GS_NORMAL:
206     pop  ACC                ; 恢复容量值
207     mov  DPTR, #SEG_TAB
208     movc A, @A+DPTR
209     ret
210
211 ;-----
212 ; GET_CALC_SEG — 计算器显示缓存转段码
213 ; 输入: A = 0-9 数字, 0x0A=空白, 0x0B='-', 0x0C='E'
214 ; 输出: A = 写入 P1 的段码
215 ;-----
216 GET_CALC_SEG:
217     cjne A, #0x0A, GCS_MINUS
218     mov  A, #0x00
219     ret
220 GCS_MINUS:
221     cjne A, #0x0B, GCS_ERR
222     mov  A, #0x02          ; 仅 g 段, 显示 '-'
223     ret
224 GCS_ERR:
225     cjne A, #0x0C, GCS_DIGIT
226     mov  A, #0x9E          ; E
227     ret
228 GCS_DIGIT:
229     mov  DPTR, #SEG_TAB
230     movc A, @A+DPTR
231     ret
232
233 ;-----
234 ; DELAY_DIG — 保持当前位约 1ms (24.5MHz 下)
235 ;-----
236 DELAY_DIG:
237     mov  R6, #8
238 DD1:
239     mov  R5, #180
240 DD2:
241     djnz R5, DD2
242     djnz R6, DD1
243     ret
244
245 ;-----
246 ; DISPLAY_BLANK_GAP — 段选关闭后的短消隐, 释放残留亮度
247 ;-----
248 DISPLAY_BLANK_GAP:
249     mov  R4, #18
250 DBG1:
251     djnz R4, DBG1
252     ret

```

Listing 7: 矩阵键盘扫描驱动：src/keypad.asm

```

1 ;-----
2 ; keypad.asm — 4x4 矩阵键盘扫描
3 ;
4 ; 硬件连接：
5 ;   行输出：P2.0-P2.3（每次拉低一行）
6 ;   列输入：P2.4-P2.7（低电平表示按下）
7 ;
8 ; 键位映射遵循 src/reference.asm。
9 ; SCAN_KEY 返回 PDF 键号 K0..K15。
10 ;
11 ; 导出函数：
12 ;   SCAN_KEY — 返回 A=PDF 键号（0-15），无键返回 0xFF
13 ;
14 ; 重映射后的业务含义：
15 ;   K1-K9      -> 批量投放数量
16 ;   K10-K13    -> F1-F4 分类桶按键
17 ;   K14        -> F5，正常模式进入计算器，计算器模式 back
18 ;   K15        -> F6，物业清空
19 ;
20 ; 会改写：ACC, R7, TMP, ROWIDX
21 ;-----
22     .include "sfr.inc"
23     .include "iram.inc"
24
25     .module keypad
26
27     .globl SCAN_KEY
28
29     .globl KEY_REMAP_TAB    ; 定义在 tables.asm
30
31     .area KEY_CODE (CODE)
32
33 ;-----
34 ; SCAN_KEY — 带消抖的完整键盘扫描
35 ; 返回 A = 重映射后的 PDF 键号（0-15），无键返回 0xFF
36 ;-----
37 SCAN_KEY:
38     mov ROWIDX, #0
39     mov P2, #0xFE          ; 第 0 行拉低
40     lcall READ_COL
41     cjne A, #0xFF, SK_GOT
42
43     mov ROWIDX, #1
44     mov P2, #0xFD          ; 第 1 行拉低
45     lcall READ_COL
46     cjne A, #0xFF, SK_GOT
47
48     mov ROWIDX, #2

```

```

49     mov P2, #0xFB          ; 第 2 行拉低
50     lcall READ_COL
51     cjne A, #0xFF, SK_GOT
52
53     mov ROWIDX, #3
54     mov P2, #0xF7          ; 第 3 行拉低
55     lcall READ_COL
56     cjne A, #0xFF, SK_GOT
57
58     mov P2, #0xFF
59     mov A, #0xFF
60     ret
61
62 SK_GOT:
63     ; 消抖: 通过显示刷新等待约 30ms 后重读同一行
64     lcall DEBOUNCE_DELAY
65
66     mov A, ROWIDX
67     cjne A, #0, DB_R1
68     mov P2, #0xFE
69     sjmp DB_READ
70 DB_R1:
71     cjne A, #1, DB_R2
72     mov P2, #0xFD
73     sjmp DB_READ
74 DB_R2:
75     cjne A, #2, DB_R3
76     mov P2, #0xFB
77     sjmp DB_READ
78 DB_R3:
79     mov P2, #0xF7
80 DB_READ:
81     lcall READ_COL
82     mov P2, #0xFF
83
84     cjne A, #0xFF, DB_OK
85     mov A, #0xFF          ; 抖动造成的无效按键
86     ret
87
88 DB_OK:
89     ; 计算扫描码 row*4+col, 再查表映射为 PDF 键号
90     mov TMP, A
91     mov A, ROWIDX
92     rl A
93     rl A
94     add A, TMP
95     mov DPTR, #KEY_REMAP_TAB
96     movc A, @A+DPTR
97     ret

```

```

98
99 ;-----
100 ; READ_COL — 读取当前行对应的列输入
101 ; 返回 A = 列号 (0-3)，无列按下返回 0xFF
102 ;-----
103 READ_COL:
104     mov A, P2
105     anl A, #0xF0
106     cjne A, #0xF0, RC_HAS
107     mov A, #0xFF
108     ret
109 RC_HAS:
110     jb  0xA4, RC1          ; P2.4
111     mov A, #0
112     ret
113 RC1:
114     jb  0xA5, RC2          ; P2.5
115     mov A, #1
116     ret
117 RC2:
118     jb  0xA6, RC3          ; P2.6
119     mov A, #2
120     ret
121 RC3:
122     jb  0xA7, RC_NONE      ; P2.7
123     mov A, #3
124     ret
125 RC_NONE:
126     mov A, #0xFF
127     ret
128
129 ;-----
130 ; DEBOUNCE_DELAY — 通过显示刷新延时约 30ms
131 ;-----
132 DEBOUNCE_DELAY:
133     mov R7, #30
134 DB1:
135     lcall REFRESH_ALL
136     djnz R7, DB1
137     ret

```

Listing 8: D1-D8 流水灯驱动：src/led_bar.asm

```

1 ;-----
2 ; led_bar.asm — D1-D8 流水灯进度条驱动（经 74HCT164）
3 ;
4 ; 硬件连接：
5 ;   74HCT164 移位寄存器：DATA=P3.3, CLK=P3.4
6 ;   Q1=D1 .. Q8=D8, 0=亮, 1=灭

```

```

7 ; D9 为独立指示灯，接 P0.0，0=亮，1=灭
8 ;
9 ; 导出函数：
10 ; UPDATE_LED_BAR — 根据桶盖倒计时计算 LEDREG，变化时移位输出
11 ; SHIFT_LED_BAR — 将 LEDREG 输出到 74HCT164（移位期间关闭中断）
12 ;
13 ; LED 级别表（LED_BAR_TAB，位于 tables.asm）：
14 ; 下标 0..8，表值为 D1..D8 位掩码（1=灭，0=亮，从高位送出）
15 ; 0 -> 0xFF（全灭），8 -> 0x00（全亮）
16 ;-----
17 .include "sfr.inc"
18 .include "iram.inc"
19
20 .module led_bar
21
22 .globl UPDATE_LED_BAR
23 .globl SHIFT_LED_BAR
24
25 .globl LED_BAR_TAB ; 定义在 tables.asm
26
27 .area LED_CODE (CODE)
28
29 ;-----
30 ; UPDATE_LED_BAR
31 ; 桶盖关闭：强制 D1-D8 全灭（若已经全灭则不重复移位）。
32 ; 桶盖打开：将剩余时间（LIDHI:LIDLO）映射为 0-8 格进度。
33 ; D9 由 init/isr 管理，开盖时点亮。
34 ;-----
35 UPDATE_LED_BAR:
36     mov A, LIDOPEN
37     jnz ULB_OPEN
38
39     ; 桶盖关闭：流水灯全灭
40     mov A, LASTBAR
41     cjne A, #0xFF, ULB_CLOSE_APPLY
42     ret ; 已经全灭，无需更新
43 ULB_CLOSE_APPLY:
44     mov LEDREG, #0xFF
45     mov LASTBAR, #0xFF
46     lcall SHIFT_LED_BAR
47     ret
48
49 ULB_OPEN:
50     ; 将 LIDHI:LIDLO（0-8000ms）映射为 0-8 格
51     ; 阈值：7000,6000,5000,4000,3000,2000,1000,1,0
52     ; 使用 16 位比较：先比较高字节，再比较低字节
53
54     ; 是否 >= 7000（0x1B58）
55     mov A, LIDHI

```

```

56     clr  C
57     subb A, #0x1B
58     jc   ULB_LE7
59     jnz  ULB_GE8
60     mov  A, LIDL0
61     clr  C
62     subb A, #0x58
63     jc   ULB_LE7
64 ULB_GE8:
65     mov  A, #8
66     ljmp ULB_IDX
67
68 ULB_LE7:
69     ; 是否 >= 6000 (0x1770)
70     mov  A, LIDHI
71     clr  C
72     subb A, #0x17
73     jc   ULB_LE6
74     jnz  ULB_GE7
75     mov  A, LIDL0
76     clr  C
77     subb A, #0x70
78     jc   ULB_LE6
79 ULB_GE7:
80     mov  A, #7
81     ljmp ULB_IDX
82
83 ULB_LE6:
84     ; 是否 >= 5000 (0x1388)
85     mov  A, LIDHI
86     clr  C
87     subb A, #0x13
88     jc   ULB_LE5
89     jnz  ULB_GE6
90     mov  A, LIDL0
91     clr  C
92     subb A, #0x88
93     jc   ULB_LE5
94 ULB_GE6:
95     mov  A, #6
96     ljmp ULB_IDX
97
98 ULB_LE5:
99     ; 是否 >= 4000 (0x0FA0)
100     mov  A, LIDHI
101     clr  C
102     subb A, #0x0F
103     jc   ULB_LE4
104     jnz  ULB_GE5

```



```

105     mov  A, LIDL0
106     clr  C
107     subb A, #0xA0
108     jc   ULB_LE4
109 ULB_GE5:
110     mov  A, #5
111     ljmp ULB_IDX
112
113 ULB_LE4:
114     ; 是否 >= 3000 (0x0BB8)
115     mov  A, LIDHI
116     clr  C
117     subb A, #0x0B
118     jc   ULB_LE3
119     jnz  ULB_GE4
120     mov  A, LIDL0
121     clr  C
122     subb A, #0xB8
123     jc   ULB_LE3
124 ULB_GE4:
125     mov  A, #4
126     ljmp ULB_IDX
127
128 ULB_LE3:
129     ; 是否 >= 2000 (0x07D0)
130     mov  A, LIDHI
131     clr  C
132     subb A, #0x07
133     jc   ULB_LE2
134     jnz  ULB_GE3
135     mov  A, LIDL0
136     clr  C
137     subb A, #0xD0
138     jc   ULB_LE2
139 ULB_GE3:
140     mov  A, #3
141     ljmp ULB_IDX
142
143 ULB_LE2:
144     ; 是否 >= 1000 (0x03E8)
145     mov  A, LIDHI
146     clr  C
147     subb A, #0x03
148     jc   ULB_LE1
149     jnz  ULB_GE2
150     mov  A, LIDL0
151     clr  C
152     subb A, #0xE8
153     jc   ULB_LE1

```

```

154 ULB_GE2:
155     mov  A, #2
156     ljmp ULB_IDX
157
158 ULB_LE1:
159     mov  A, LIDHI
160     orl  A, LIDLO
161     jz   ULB_ZERO
162     mov  A, #1
163     ljmp ULB_IDX
164 ULB_ZERO:
165     mov  A, #0
166
167 ULB_IDX:
168     ; A = 级别 0-8, 查表得到位掩码
169     mov  DPTR, #LED_BAR_TAB
170     movc A, @A+DPTR
171
172 ULB_APPLY:
173     mov  LEDREG, A
174     xrl  A, LASTBAR
175     jz   ULB_RET          ; 无变化, 跳过移位
176     mov  A, LEDREG
177     mov  LASTBAR, A
178     lcall SHIFT_LED_BAR
179 ULB_RET:
180     ret
181
182 ;-----
183 ; SHIFT_LED_BAR — 将 LEDREG 从最高位开始送入 74HCT164
184 ; 移位期间关闭中断, 避免时钟被打断。
185 ;-----
186 SHIFT_LED_BAR:
187     clr  EA                ; 关闭中断
188     mov  A, LEDREG
189     mov  R0, #8
190 SLB_LP:
191     mov  C, ACC.7
192     mov  LEDDAT, C          ; P3.3 = 数据位
193     setb LEDCLK             ; P3.4 产生上升沿
194     nop
195     clr  LEDCLK
196     rl   A
197     djnz R0, SLB_LP
198     setb EA                ; 重新打开中断
199     ret

```

Listing 9: ROM 查找表: src/tables.asm

```

1 ;-----
2 ; tables.asm — ROM 查找表
3 ; SEG_TAB      : 数字 0-9 的数码管段码
4 ; LED_BAR_TAB  : D1-D8 进度级别 0-8 的位掩码
5 ; KEY_REMAP_TAB : 物理扫描码 -> PDF 键号 K0..K15
6 ;-----
7     .module tables
8
9     .globl SEG_TAB
10    .globl LED_BAR_TAB
11    .globl KEY_REMAP_TAB
12
13    .area TABLES (CODE)
14
15 ; 数码管段码, 共阴, 高电平点亮
16 ; 位序: a=P1.7, b=P1.6, c=P1.5, d=P1.4, e=P1.3, f=P1.2, g=P1.1, dp=P1.0
17 ; 数字 0-9
18 SEG_TAB:
19     .byte 0xFC    ; 0: abcdef
20     .byte 0x60    ; 1: bc
21     .byte 0xDA    ; 2: abdeg
22     .byte 0xF2    ; 3: abcdg
23     .byte 0x66    ; 4: bcfg
24     .byte 0xB6    ; 5: acdfg
25     .byte 0xBE    ; 6: acdefg
26     .byte 0xE0    ; 7: abc
27     .byte 0xFE    ; 8: abcdefg
28     .byte 0xF6    ; 9: abcdfg
29
30 ; 74HCT164 流水灯位掩码 (1=灭, 0=亮)
31 ; 下标 = 从 D1 开始点亮的 LED 数量 (0=全灭, 8=全亮)
32 LED_BAR_TAB:
33     .byte 0xFF    ; 0: 全灭
34     .byte 0xFE    ; 1: D1 亮
35     .byte 0xFC    ; 2: D1-D2 亮
36     .byte 0xF8    ; 3: D1-D3 亮
37     .byte 0xF0    ; 4: D1-D4 亮
38     .byte 0xE0    ; 5: D1-D5 亮
39     .byte 0xC0    ; 6: D1-D6 亮
40     .byte 0x80    ; 7: D1-D7 亮
41     .byte 0x00    ; 8: 全亮
42
43 ; 物理扫描码 (row*4+col, 0-15) -> PDF 键号 K0..K15。
44 ; 此表来自 src/reference.asm。
45 ; PDF 键号:
46 ; K10..K13 = F1..F4, K14 = F5, K15 = F6
47 KEY_REMAP_TAB:
48     .byte 0, 4, 8, 12
49     .byte 1, 5, 9, 13

```

```

50     .byte 2, 6,10, 14
51     .byte 3, 7,11, 15

```

5.6 业务状态机与扩展功能

Listing 10: 垃圾分类主状态机：src/main.asm

```

1  ;-----
2  ; main.asm — 主循环与垃圾桶状态机
3  ;
4  ; 按键逻辑：
5  ;   K10-K13 (F1-F4) 选择 0-3 号垃圾桶
6  ;   桶满（容量为 0）时蜂鸣 1 秒，并显示 F
7  ;   桶未满且桶盖关闭时，开盖 8 秒，D9 点亮
8  ;   开盖期间按同一个 F 键或 K1-K9 进行投放
9  ;   Kint 键提前关盖，由 EXT0_ISR 处理
10 ;   K15 (F6) 为物业清空，将所有容量恢复为 9
11 ;   K14 (F5) 进入计算器模式，Kint 退出计算器模式
12 ;
13 ; SCAN_KEY 返回 reference.asm 中的 PDF 键号：
14 ;   K0-K9 为数字键，K10-K13 为 F1-F4，K14 为 F5，K15 为 F6
15 ;-----
16     .include "sfr.inc"
17     .include "iram.inc"
18
19     .module main
20
21     .globl MAIN_LOOP
22     .globl ENTER_CALC
23     .globl CALC_HANDLE_KEY
24
25     .area MAIN_CODE (CODE)
26
27 ;-----
28 ; 主循环
29 ;-----
30 MAIN_LOOP:
31     lcall UPDATE_LED_BAR
32
33     ; 每次键盘扫描之间刷新显示约 40 次（约 40ms）
34     mov  CNT, #40
35 ML_REFRESH:
36     lcall REFRESH_ALL
37     djnz CNT, ML_REFRESH
38
39     lcall SCAN_KEY           ; A = 重映射后的 PDF 键号，0xFF 表示无键
40     mov  KEYNOW, A
41

```

```

42     cjne A, #0xFF, ML_HAS_KEY
43     mov  KEYLAST, #0xFF
44     sjmp MAIN_LOOP
45
46 ML_HAS_KEY:
47     ; 边沿检测：忽略持续按住的按键
48     mov  A, KEYNOW
49     cjne A, KEYLAST, ML_NEW_KEY
50     sjmp MAIN_LOOP
51
52 ML_NEW_KEY:
53     mov  KEYLAST, A
54
55     ; 计算器模式下，键盘改由计算器状态机解释。
56     mov  A, CALCMODE
57     jz   ML_NORMAL_KEY
58     lcall CALC_HANDLE_KEY
59     sjmp MAIN_LOOP
60
61 ML_NORMAL_KEY:
62     mov  A, KEYNOW
63
64     ; K15/F6：物业清空所有垃圾桶
65     cjne A, #15, ML_CHECK_CAT
66     lcall PROP_CLEAR
67     sjmp MAIN_LOOP
68
69 ML_CHECK_CAT:
70     ; K14/F5：进入计算器模式
71     mov  A, KEYNOW
72     cjne A, #14, ML_CHECK_NUM
73     lcall ENTER_CALC
74     sjmp MAIN_LOOP
75
76 ML_CHECK_NUM:
77     ; K1-K9：批量投放袋数，仅在开盖时有效。
78     ; K0 不作为投放数量。
79     mov  A, KEYNOW
80     jz   MAIN_LOOP
81     clr  C
82     subb A, #10
83     jnc  ML_CHECK_FUNC
84     lcall HANDLE_NUMBER
85     sjmp MAIN_LOOP
86
87 ML_CHECK_FUNC:
88     ; K10-K13/F1-F4：减 10 得到按键序号，再按数码管位选方向换算桶号。
89     mov  A, KEYNOW
90     clr  C

```

```

91     subb A, #10
92     mov  TMP, A
93     clr  C
94     subb A, #4
95     jc   ML_CAT_KEY
96     sjmp MAIN_LOOP
97
98 ML_CAT_KEY:
99     mov  A, #3
100    clr  C
101    subb A, TMP
102    mov  TMP, A
103    lcall HANDLE_CATEGORY
104    sjmp MAIN_LOOP
105
106 ;-----
107 ; HANDLE_CATEGORY — 处理 F1-F4 分类键, TMP 中为桶号
108 ;-----
109 HANDLE_CATEGORY:
110     ; 判断桶盖是否已经打开
111     mov  A, LIDOPEN
112     jz   HC_LID_CLOSED
113
114     ; 桶盖已打开时, 只有再次按同一类才投放 1 袋
115     mov  A, TMP
116     cjne A, SELBIN, HC_RET
117     mov  R2, #1
118     lcall HANDLE_BAGS
119 HC_RET:
120     ret
121
122 HC_LID_CLOSED:
123     ; 检查容量
124     lcall GET_CAP           ; A = TMP 对应桶的容量
125     jnz  HC_OPEN_LID
126
127     ; 桶满: 蜂鸣并显示 F 约 1 秒
128     lcall FULL_ALERT
129     ret
130
131 HC_OPEN_LID:
132     ; 选中桶未满: 开盖并启动 8 秒倒计时
133     mov  A, TMP
134     mov  SELBIN, A
135     mov  FULLBIN, #0xFF
136     mov  ERRBIN, #0xFF
137     mov  LIDOPEN, #1
138     clr  EA
139     mov  LIDL0, #0x40      ; 8000ms = 0x1F40

```

```

140     mov  LIDHI, #0x1F
141     setb EA
142     mov  BLKDIV, #0
143     mov  BLKFLG, #0
144     clr  D9_LED           ; D9 点亮
145     ret
146
147 ;-----
148 ; HANDLE_NUMBER — 处理 K1-K9 批量投放, KEYNOW 中为袋数
149 ;-----
150 HANDLE_NUMBER:
151     mov  A, LIDOPEN
152     jz   HN_RET
153     mov  A, SELBIN
154     cjne A, #0xFF, HN_HAVE_BIN
155     ret
156 HN_HAVE_BIN:
157     mov  A, KEYNOW
158     mov  R2, A
159     lcall HANDLE_BAGS
160 HN_RET:
161     ret
162
163 ;-----
164 ; HANDLE_BAGS — 若容量足够, 向 SELBIN 投放 R2 袋。
165 ; 容量不足时显示 E、蜂鸣并立即关盖。
166 ;-----
167 HANDLE_BAGS:
168     mov  A, SELBIN
169     cjne A, #0xFF, HB_HAVE_BIN
170     ret
171 HB_HAVE_BIN:
172     mov  R3, A           ; 保存桶号
173     add  A, #CAPO
174     mov  R0, A
175     mov  A, @R0
176     clr  C
177     subb A, R2
178     jc   HB_ERROR
179     mov  @R0, A
180     mov  FULLBIN, #0xFF
181     mov  ERRBIN, #0xFF
182     ret
183 HB_ERROR:
184     mov  A, R3
185     mov  ERRBIN, A
186     mov  FULLBIN, #0xFF
187     clr  EA
188     mov  HINTLO, #0xE8

```

```

189     mov  HINTHI, #0x03
190     setb EA
191     setb BUZZER
192     mov  LIDOPEN, #0
193     mov  SELBIN,  #0xFF
194     mov  LIDL0,   #0
195     mov  LIDHI,   #0
196     setb D9_LED
197     ret
198
199 ;-----
200 ; GET_CAP — 读取 TMP 指定桶的容量
201 ; 返回: A = 容量 (0-9)
202 ;-----
203 GET_CAP:
204     mov  A, TMP
205     cjne A, #0, GC1
206     mov  A, CAP0
207     ret
208 GC1:
209     cjne A, #1, GC2
210     mov  A, CAP1
211     ret
212 GC2:
213     cjne A, #2, GC3
214     mov  A, CAP2
215     ret
216 GC3:
217     mov  A, CAP3
218     ret
219
220 ;-----
221 ; FULL_ALERT — 蜂鸣约 1 秒, 并在 TMP 指定数码管显示 F
222 ;-----
223 FULL_ALERT:
224     mov  A, TMP
225     mov  FULLBIN, A
226     mov  ERRBIN, #0xFF
227     setb BUZZER
228     clr  EA
229     mov  HINTLO, #0xE8      ; 1000ms = 0x03E8
230     mov  HINTHI, #0x03
231     setb EA
232     ret
233
234 ;-----
235 ; PROP_CLEAR — F6: 所有垃圾桶容量恢复为 9
236 ;-----
237 PROP_CLEAR:

```



```

238     mov  CAP0, #9
239     mov  CAP1, #9
240     mov  CAP2, #9
241     mov  CAP3, #9
242     mov  FULLBIN, #0xFF
243     mov  ERRBIN, #0xFF
244     mov  LIDOPEN, #0
245     mov  SELBIN, #0xFF
246     mov  LIDL0, #0
247     mov  LIDHI, #0
248     setb D9_LED
249     mov  LEDREG, #0xFF
250     mov  LASTBAR, #0
251     lcall SHIFT_LED_BAR
252     setb BUZZER
253     clr  EA
254     mov  HINTLO, #0xE8
255     mov  HINTHI, #0x03
256     setb EA
257     ret

```

Listing 11: 计算器扩展状态机：src/calculator.asm

```

1  ;-----
2  ; calculator.asm — 计算器模式状态机
3  ;
4  ; 正常模式下 K14/KF5 进入计算器模式。
5  ; 计算器模式按键：
6  ;   K0-K9      输入数字（每个操作数最多两位）
7  ;   K10-K13    KF1-KF4，对应 + - * /
8  ;   K14        KF5, Back
9  ;   K15        KF6, 等号
10 ;   Kint       由 EXTO_ISR 处理，退出计算器模式
11 ;-----
12     .include "sfr.inc"
13     .include "iram.inc"
14
15     .module calculator
16
17     .globl ENTER_CALC
18     .globl CALC_HANDLE_KEY
19
20     .globl SHIFT_LED_BAR
21
22     .area CALC_CODE (CODE)
23
24 ;-----
25 ; ENTER_CALC — K14/KF5：进入计算器模式
26 ;-----

```

```

27 ENTER_CALC:
28     clr  EA
29     mov  LIDOPEN, #0
30     mov  SELBIN,  #0xFF
31     mov  LIDL0,   #0
32     mov  LIDHI,   #0
33     mov  HINTLO,  #0
34     mov  HINTHI,  #0
35     setb EA
36
37     clr  BUZZER
38     setb D9_LED
39     mov  FULLBIN, #0xFF
40     mov  ERRBIN,  #0xFF
41     mov  LEDREG,  #0xFF
42     mov  LASTBAR, #0
43     lcall SHIFT_LED_BAR
44
45     mov  CALCMODE, #1
46     lcall CALC_RESET
47     ret
48
49 ;-----
50 ; CALC_RESET — 清空计算器输入和显示缓存
51 ;-----
52 CALC_RESET:
53     mov  CALCSTATE, #0
54     mov  CALCOP,     #0
55     mov  OP1,        #0
56     mov  OP2,        #0
57     mov  DIGCNT,     #0
58     mov  CALCNEG,    #0
59     mov  RESLO,      #0
60     mov  RESHI,      #0
61     lcall CALC_CLEAR_DISP
62     ret
63
64 CALC_CLEAR_DISP:
65     mov  CALCD0, #0x0A
66     mov  CALCD1, #0x0A
67     mov  CALCD2, #0x0A
68     mov  CALCD3, #0x0A
69     ret
70
71 ;-----
72 ; CALC_HANDLE_KEY — 计算器模式按键分发
73 ;-----
74 CALC_HANDLE_KEY:
75     mov  A, KEYNOW

```

```

76     cjne A, #14, CHK_EQ
77     lcall CALC_BACK
78     ret
79
80 CHK_EQ:
81     cjne A, #15, CHK_DIGIT
82     lcall CALC_EQUALS
83     ret
84
85 CHK_DIGIT:
86     clr C
87     subb A, #10
88     jnc CHK_OP
89     lcall CALC_INPUT_DIGIT
90     ret
91
92 CHK_OP:
93     mov A, KEYNOW
94     clr C
95     subb A, #10
96     mov TMP, A
97     clr C
98     subb A, #4
99     jc  CHK_OP_OK
100    ret
101 CHK_OP_OK:
102    lcall CALC_INPUT_OP
103    ret
104
105 ;-----
106 ; CALC_INPUT_DIGIT — 输入 0-9, 单个操作数最多两位
107 ;-----
108 CALC_INPUT_DIGIT:
109     mov A, CALCSTATE
110     cjne A, #3, CID_NOT_RESULT
111     lcall CALC_RESET      ; 结果后输入数字, 开始下一轮
112 CID_NOT_RESULT:
113     mov A, CALCSTATE
114     cjne A, #1, CID_HAVE_SIDE
115     mov CALCSTATE, #2
116     mov DIGCNT, #0
117     mov OP2, #0
118     lcall CALC_CLEAR_DISP
119
120 CID_HAVE_SIDE:
121     mov A, DIGCNT
122     cjne A, #2, CID_ROOM
123     ret
124

```

```

125 CID_ROOM:
126     mov  A, CALCSTATE
127     cjne A, #2, CID_OP1
128     mov  R0, #OP2
129     sjmp CID_APPEND
130 CID_OP1:
131     mov  R0, #OP1
132
133 CID_APPEND:
134     mov  A, @R0
135     mov  B, #10
136     mul  AB
137     add  A, KEYNOW
138     mov  @R0, A
139     inc  DIGCNT
140     lcall CALC_SHOW_INPUT
141     ret
142
143 ;-----
144 ; CALC_INPUT_OP — 输入 KF1-KF4 运算符
145 ;   KF1=+, KF2=-, KF3=*, KF4=/
146 ;-----
147 CALC_INPUT_OP:
148     mov  A, CALCSTATE
149     cjne A, #1, CIO_NEED_OP1
150     sjmp CIO_SET
151 CIO_NEED_OP1:
152     mov  A, CALCSTATE
153     jnz  CIO_RET
154     mov  A, DIGCNT
155     jz   CIO_RET
156 CIO_SET:
157     mov  A, TMP
158     inc  A                ; TMP 0-3 -> CALCOP 1-4
159     mov  CALCOP, A
160     mov  CALCSTATE, #1
161     mov  DIGCNT, #0
162     mov  OP2, #0
163     lcall CALC_SHOW_OP
164 CIO_RET:
165     ret
166
167 ;-----
168 ; CALC_BACK — KF5: 退格/返回上一步
169 ;-----
170 CALC_BACK:
171     mov  A, CALCSTATE
172     cjne A, #3, CB_NOT_RESULT
173     lcall CALC_RESET

```

```

174     ret
175
176 CB_NOT_RESULT:
177     mov  A, CALCSTATE
178     cjne A, #1, CB_NOT_OP
179     mov  CALCSTATE, #0
180     mov  A, OP1
181     clr  C
182     subb A, #10
183     jc   CB_OP1_ONE
184     mov  DIGCNT, #2
185     sjmp CB_SHOW_OP1
186 CB_OP1_ONE:
187     mov  DIGCNT, #1
188 CB_SHOW_OP1:
189     lcall CALC_SHOW_INPUT
190     ret
191
192 CB_NOT_OP:
193     mov  A, DIGCNT
194     jnz  CB_DEL_DIGIT
195     mov  A, CALCSTATE
196     cjne A, #2, CB_RET
197     mov  CALCSTATE, #1
198     lcall CALC_SHOW_OP
199 CB_RET:
200     ret
201
202 CB_DEL_DIGIT:
203     mov  A, CALCSTATE
204     cjne A, #2, CB_DEL_OP1
205     mov  R0, #OP2
206     sjmp CB_DEL_DO
207 CB_DEL_OP1:
208     mov  R0, #OP1
209 CB_DEL_DO:
210     mov  A, @R0
211     mov  B, #10
212     div  AB
213     mov  @R0, A
214     dec  DIGCNT
215     lcall CALC_SHOW_INPUT
216     ret
217
218 ;-----
219 ; CALC_SHOW_INPUT — 右对齐显示当前操作数
220 ;-----
221 CALC_SHOW_INPUT:
222     lcall CALC_CLEAR_DISP

```

```

223     mov  A, DIGCNT
224     jnz  CSI_HAVE
225     ret
226 CSI_HAVE:
227     mov  A, CALCSTATE
228     cjne A, #2, CSI_OP1
229     mov  A, OP2
230     sjmp CSI_SPLIT
231 CSI_OP1:
232     mov  A, OP1
233 CSI_SPLIT:
234     mov  B, #10
235     div  AB           ; A=十位, B=个位
236     mov  R4, A
237     mov  R5, B
238     mov  A, DIGCNT
239     cjne A, #1, CSI_TWO
240     mov  CALCD3, R5
241     ret
242 CSI_TWO:
243     mov  CALCD2, R4
244     mov  CALCD3, R5
245     ret
246
247 ;-----
248 ; CALC_SHOW_OP — 运算符提示: KF1-KF4 对应第 1-4 位显示 8
249 ;-----
250 CALC_SHOW_OP:
251     lcall CALC_CLEAR_DISP
252     mov  A, CALCOP
253     cjne A, #1, CSO_SUB
254     mov  CALCD0, #8
255     ret
256 CSO_SUB:
257     cjne A, #2, CSO_MUL
258     mov  CALCD1, #8
259     ret
260 CSO_MUL:
261     cjne A, #3, CSO_DIV
262     mov  CALCD2, #8
263     ret
264 CSO_DIV:
265     mov  CALCD3, #8
266     ret
267
268 ;-----
269 ; CALC_EQUALS — KF6: 计算并显示结果
270 ;-----
271 CALC_EQUALS:

```

```

272     mov  A, CALCSTATE
273     cjne A, #2, CE_RET
274     mov  A, DIGCNT
275     jz   CE_RET
276
277     mov  CALCSTATE, #3
278     mov  CALCNEG, #0
279     mov  RESHI, #0
280     mov  RESLO, #0
281
282     mov  A, CALCOP
283     cjne A, #1, CE_SUB
284
285     ; 加法: 0-99 + 0-99
286     mov  A, OP1
287     add  A, OP2
288     mov  RESLO, A
289     mov  A, #0
290     addc A, #0
291     mov  RESHI, A
292     lcall CALC_SHOW_RESULT
293     ret
294
295 CE_SUB:
296     cjne A, #2, CE_MUL
297     mov  A, OP1
298     clr  C
299     subb A, OP2
300     jc   CE_SUB_NEG
301     mov  RESLO, A
302     lcall CALC_SHOW_RESULT
303     ret
304 CE_SUB_NEG:
305     mov  CALCNEG, #1
306     mov  A, OP2
307     clr  C
308     subb A, OP1
309     mov  RESLO, A
310     lcall CALC_SHOW_RESULT
311     ret
312
313 CE_MUL:
314     cjne A, #3, CE_DIV
315     mov  A, OP1
316     mov  B, OP2
317     mul  AB
318     mov  RESLO, A
319     mov  RESHI, B
320     lcall CALC_SHOW_RESULT

```

```

321     ret
322
323 CE_DIV:
324     mov  A, OP2
325     jnz  CE_DIV_OK
326     lcall CALC_SHOW_ERROR
327     ret
328 CE_DIV_OK:
329     mov  A, OP1
330     mov  B, OP2
331     div  AB           ; A=商, B=余数
332     mov  R4, A
333     mov  R5, B
334     lcall CALC_SHOW_DIV
335 CE_RET:
336     ret
337
338 ;-----
339 ; CALC_SHOW_RESULT — 显示非除法结果, 支持 -99..9801
340 ;-----
341 CALC_SHOW_RESULT:
342     lcall CALC_CLEAR_DISP
343     mov  A, CALCNEG
344     jz   CSR_POS
345
346     mov  CALCD0, #0x0B
347     mov  A, RESLO
348     mov  B, #10
349     div  AB
350     mov  R4, A
351     mov  R5, B
352     mov  A, R4
353     jz   CSR_NEG_ONE
354     mov  CALCD2, R4
355 CSR_NEG_ONE:
356     mov  CALCD3, R5
357     ret
358
359 CSR_POS:
360     mov  R4, #0           ; 千位
361     mov  R5, #0           ; 百位
362     mov  R6, #0           ; 十位
363
364 CSR_THOUS:
365     mov  A, RESHI
366     clr  C
367     subb A, #0x03
368     jc   CSR_HUNDS
369     jnz  CSR_SUB_THOUS

```



```

370     mov  A, RESLO
371     clr  C
372     subb A, #0xE8
373     jc   CSR_HUNDS
374 CSR_SUB_THOUS:
375     mov  A, RESLO
376     clr  C
377     subb A, #0xE8
378     mov  RESLO, A
379     mov  A, RESHI
380     subb A, #0x03
381     mov  RESHI, A
382     inc  R4
383     sjmp CSR_THOUS
384
385 CSR_HUNDS:
386     mov  A, RESHI
387     jnz  CSR_SUB_HUND
388     mov  A, RESLO
389     clr  C
390     subb A, #100
391     jc   CSR_TENS
392 CSR_SUB_HUND:
393     mov  A, RESLO
394     clr  C
395     subb A, #100
396     mov  RESLO, A
397     mov  A, RESHI
398     subb A, #0
399     mov  RESHI, A
400     inc  R5
401     sjmp CSR_HUNDS
402
403 CSR_TENS:
404     mov  A, RESLO
405     clr  C
406     subb A, #10
407     jc   CSR_ONES
408     mov  RESLO, A
409     inc  R6
410     sjmp CSR_TENS
411
412 CSR_ONES:
413     mov  A, R4
414     jz   CSR_HUND_DIG
415     mov  CALCD0, R4
416 CSR_HUND_DIG:
417     mov  A, R4
418     orl  A, R5

```

```

419     jz    CSR_TEN_DIG
420     mov   CALCD1, R5
421 CSR_TEN_DIG:
422     mov   A, R4
423     orl   A, R5
424     orl   A, R6
425     jz    CSR_ONE_DIG
426     mov   CALCD2, R6
427 CSR_ONE_DIG:
428     mov   CALCD3, RESLO
429     ret
430
431 ;-----
432 ; CALC_SHOW_DIV — 除法显示：前两位商，后两位余数
433 ;-----
434 CALC_SHOW_DIV:
435     mov   A, R4
436     mov   B, #10
437     div   AB
438     mov   CALCD0, A
439     mov   CALCD1, B
440     mov   A, R5
441     mov   B, #10
442     div   AB
443     mov   CALCD2, A
444     mov   CALCD3, B
445     ret
446
447 CALC_SHOW_ERROR:
448     mov   CALCD0, #0x0C
449     mov   CALCD1, #0x0C
450     mov   CALCD2, #0x0C
451     mov   CALCD3, #0x0C
452     ret

```

6 程序测试方法与结果

本节对程序的构建结果和板级功能进行测试。测试平台为 C8051F310EVM，观察对象包括四位数码管、D1–D8 流水灯、D9 桶盖状态灯和蜂鸣器，测试内容覆盖基础要求、提高要求和个性化扩展功能。

6.1 编译测试

本工程采用 sdas8051 汇编器和 sld 链接器构建，执行工程构建脚本后，9 个汇编源文件均成功生成目标文件，并链接得到 c8051f310.hex。当前构建输出无 Error，未

出现 Warning 提示, 生成的 Intel HEX 文件大小约为 5.0KB。

根据链接生成的 c8051f310.map 文件统计, 程序各代码段和查表数据段实际占用约 2111 字节, 最高使用代码地址约为 0x0B7A, 低于 C8051F310 本实验可用的 8KB 程序空间限制, 容量满足下载和运行要求。

6.2 功能测试项目

测试项目	操作方法	预期现象
上电初始化	烧录程序后复位实验板	四位数码管显示 4 个 0 ~ 9 的容量值, 蜂鸣器关闭, D9 熄灭, D1-D8 流水灯熄灭。
F1-F4 开盖	按下 K10-K13(F1-F4)中的任一分类键	若对应容量大于 0, 则 D9 点亮, 桶盖进入 8s 开启状态, 选中数码管以约 5Hz 闪烁, 流水灯开始显示倒计时进度。
单袋投递	开盖后再次按下同一分类键	当前垃圾桶容量减少 1, 其它垃圾桶容量保持不变。
数字键多袋投递	开盖后按 K1-K9	当前垃圾桶容量按数字键值一次性减少对应袋数。
超量投递	开盖后按下大于剩余容量的数字键	容量不变, 对应数码管显示 E, 蜂鸣器鸣响约 1s, 并立即关闭桶盖。
满桶拒绝	将某桶容量投至 0 后, 再按对应 F 键	系统不开盖, 对应数码管显示 F, 蜂鸣器鸣响约 1s 后恢复显示 0。
自动关盖	开盖后不再操作	约 8s 后自动关盖, D9 熄灭, D1-D8 进度条结束, 数码管停止闪烁。
提前关盖	开盖期间按 Kint	外部中断立即清除开盖倒计时, D9 熄灭, 流水灯全灭, 系统返回待机状态。
物业清空	按 K15 (F6)	四个垃圾桶容量恢复为 9, 当前开盖状态被清除, 蜂鸣器短响确认。
计算器模式	按 K14 (F5) 进入, 使用 K0-K9 和 K10-K15 操作	数码管切换为计算器显示, 支持数字输入、四则运算、退格和等号; 按 Kint 后退出并恢复垃圾分类显示。

6.3 测试结果

按照表中项目逐项测试后，系统表现与预期一致。上电或复位后，四个垃圾桶容量能够正常初始化并显示；按下 F1–F4 分类键后，容量非零的垃圾桶能够正常开盖，D9、流水灯和选中数码管闪烁状态均能正确联动。开盖期间，同类键单袋投递和 K1–K9 数字键多袋投递均能正确扣减容量。

异常场景测试中，满桶状态下系统能够拒绝开盖并显示 F；批量投放超过剩余容量时，系统能够显示 E、蜂鸣提示并立即关闭桶盖，且容量保持不变。定时测试中，8s 自动关盖准确发生；外部中断测试中，Kint 能够立即提前关盖，说明 Timer0 中断和 INT0 中断能够与主循环状态机正确配合。

扩展功能测试中，K15 (F6) 物业清空能够将四个容量统一恢复为 9；K14 (F5) 计算器模式能够正常进入和退出，并完成基本四则运算。综合测试表明，当前程序已实现实验任务要求的基础功能和提高功能，运行过程中未发现明显显示冲突、误触发或状态无法恢复的问题。

7 实验问题分析与对策

1. 矩阵键盘物理扫描顺序与实验按键编号不一致：

- **原因分析：**4×4 矩阵键盘按行列扫描时，程序首先得到的是“行号乘以 4 再加列号”的自然扫描码。该顺序与实验资料中标注的 K0–K15 逻辑编号并不完全一致，如果直接使用扫描码，会导致 K10–K13、K14、K15 等功能键与实际按键含义错位，从而影响垃圾桶分类选择、计算器入口和物业清空功能。
- **解决方法：**在键盘扫描模块中增加 KEY_REMAP_TAB 查表映射。程序完成行列扫描和消抖后，先计算自然扫描码，再通过该表转换为实验 PDF 中的逻辑键号，使 K10–K13 正确对应 F1–F4，K14 对应 F5，K15 对应 F6。主循环只处理重映射后的键号，从而避免业务逻辑与物理键盘布局耦合。

2. 机械按键抖动与长按重复触发：

- **原因分析：**矩阵键盘为机械按键，按下和释放瞬间会出现电平抖动；同时主循环会周期性扫描键盘，如果不区分“刚按下”和“持续按住”，长按某个分类键或数字键可能被识别为多次输入，造成容量连续扣减或功能重复触发。
- **解决方法：**在 SCAN_KEY 中加入约 30ms 的消抖等待，并在主循环中使用 KEYNOW 与 KEYLAST 做边沿检测。只有当前键号与上一轮不同，且不是无键状

态时，才认为产生一次新按键事件；当扫描结果为无键时，再将 KEYLAST 恢复为 0xFF。这样既能过滤抖动，也能避免长按期间反复触发。

3. 中断与主循环共享 16 位状态变量：

- **原因分析：**Timer0 中断每 1ms 修改开盖倒计时 LIDHI:LIDLO、提示倒计时 HINTHI:HINTLO 和闪烁标志；主循环在开盖、满桶报警、超量投放和物业清空时也会写这些变量。由于 8051 写 16 位变量需要分两次写高低字节，若写入过程中被 Timer0 中断打断，可能造成倒计时高低字节短暂不一致。
- **解决方法：**在主循环需要重新装载或清零 16 位倒计时变量时，先短暂关闭总中断 EA，完成低字节和高字节写入后再打开 EA。中断服务程序内部只按固定顺序递减变量，主循环与中断之间通过这种短临界区方式避免读写冲突。

4. 容量不足时的状态恢复问题：

- **原因分析：**数字键批量投放时，如果输入袋数超过当前垃圾桶剩余容量，仅显示 E 而继续保持开盖状态，会让用户误以为仍可继续投放，与“拒绝本次投放”的业务含义不一致，也容易造成后续按键状态混乱。
- **解决方法：**在 HANDLE_BAGS 中检测到容量不足后，程序设置 ERBIN 显示 E，启动 1s 蜂鸣提示，并立即清除 LIDOPEN、SELBIN 和 LIDHI:LIDLO，同时熄灭 D9。这样系统在错误提示结束后自然回到关盖待机状态，容量值保持不变。

5. 动态数码管刷新时出现重影风险：

- **原因分析：**四位数码管采用分时动态扫描，如果在切换位选时 P1 段码仍保持上一位的输出，可能在极短时间内把上一位段码显示到下一位，表现为轻微重影。开盖闪烁、F/E 覆盖显示和计算器显示复用同一套数码管驱动时，这一问题更明显。
- **解决方法：**在 REFRESH_ALL 中切换每一位前先将 P1 清零关闭段选，并调用短消隐延时 DISPLAY_BLANK_GAP，待位选地址稳定后再输出新的段码。显示覆盖优先级统一放在 GET_SEG 中处理，保证 E、F、闪烁空白和正常数字不会互相覆盖。

6. 流水灯串行移位输出可能被中断打断：

- **原因分析:** D1–D8 流水灯由 74HCT164 串行移位寄存器驱动, 程序需要连续输出 8 位数据和时钟脉冲。如果移位过程中发生 Timer0 中断, P3.3/P3.4 的数据和时钟时序可能被拉长, 造成流水灯状态短暂异常。
- **解决方法:** 在 SHIFT_LED_BAR 中移位输出前关闭总中断 EA, 连续完成 8 位数据发送后再打开中断。由于该过程很短, 不会明显影响 1ms 系统节拍, 却能保证 74HCT164 获得完整稳定的移位时序。

本人承诺: 本报告内容真实, 无伪造数据, 无抄袭他人成果。本人完全了解学校相关规定, 如若违反, 愿意承担其后果。

签字:

陈恪瑾

2026 年 4 月 26 日